Objetivos:

I. Preparação do ambiente de desenvolvimento.

II. Exemplo prático

I. Preparação do ambiente de desenvolvimento

Passo 1 - Criar um projeto React TS com Vite

Para reproduzir os exemplos, crie um projeto React TS usando a ferramenta de build Vite:

```
npm create vite@latest front -- --template react-ts
```

Escolha o freamework: **React** e a variant: **TypeScript**

```
cd front
npm install
```

Passo 2 - Instalar as dependências do Redux

```
npm i @reduxjs/toolkit react-redux
```

Explicação dos pacotes:

- `@reduxjs/toolkit`: fornece ferramentas e convenções para simplificar o uso do Redux;
- react-redux: faz a integração entre Redux e React, permitindo que os componentes acessem o estado da Store.

Passo 3 – Criar a estrutura do Redux

Vamos organizar os arquivos do Redux em uma estrutura clara e modular. Geralmente, utilizamos uma pasta chamada store ou redux na raiz do projeto, contendo:

- store.ts: configuração da Store;
- slices: pasta para armazenar os "slices" (fatias do estado) criados com o Redux Toolkit;
- Outros arquivos: para middleware, seletores ou ações personalizadas, se necessário.

Passo 4 – Configurar a Store

A Store é o local onde o estado global da aplicação será armazenado.

Exemplo de configuração da Store (arquivo: src/redux/store.ts):

```
import { configureStore } from '@reduxjs/toolkit';
import userReducer from './slices/userSlice';

export const store = configureStore({
  reducer: {
    userObject: userReducer,
  },
});

// Tipos para uso com TypeScript
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

Explicação:

- `configureStore`: simplifica a criação da Store e já inclui ferramentas como Redux DevTools e middleware padrão;
- Reducers: são combinados em um objeto `reducer` para organizar o estado em diferentes "fatias" (slices);
- A store é criada utilizando `configureStore`. Nela, passamos um mapa de reducers, onde cada `chave` é um "slice" do estado global e cada valor é um reducer correspondente;
- reducer:
  - A chave `userObject` será usada como o identificador do estado controlado pelo userReducer;
  - No estado global, o Redux armazenará os dados gerenciados pelo userReducer sob a chave userObject.

Passo 5 – Criar um Slice com Redux Toolkit

Um slice é uma fatia do estado gerenciada por um Reducer e suas respectivas Actions.

Exemplo de um Slice (arquivo: src/redux/slices/useSlice.ts). O código define um slice do Redux chamado userSlice, que gerencia o estado de um cadastro de usuários:

```
import { createSlice } from "@reduxjs/toolkit";
import type { PayloadAction } from "@reduxjs/toolkit";

export interface User {
  id: number;
  name: string;
  age: number;
}
```

```
interface UserState {
  users: User[];
}

const userSlice = createSlice({
  name: "cadastro",
  initialState: {
    users: [],
  } as UserState,
  reducers: {
    addUser: (state, action: PayloadAction<User>) => {
      state.users.push(action.payload);
    },
    removeUser: (state, action: PayloadAction<number>) => {
      // Filtra o array e atualiza o estado
      state.users= state.users.filter((user) => action.payload !== user.id);
    },
    incrementAge: (state, action: PayloadAction<number>) => {
      // Filtra o array, mantendo apenas os usuários com id diferente do user.id
      const user = state.users.find((user) => action.payload === user.id);
      if( user ){
        user.age = user.age + 1; // Incrementa a idade no estado
      }
    }
  },
});

export const { addUser, removeUser, incrementAge } = userSlice.actions;
export default userSlice.reducer;
```

Explicação:

- UserState: representa o estado completo gerenciado por este slice. É composto de um array chamado users, que contém objetos do tipo User;
- name: identifica o slice, neste caso "cadastro". Esse nome será usado ao criar as actions e no estado global;
- initialState: define o estado inicial do slice. Aqui, o estado começa como um array vazio de usuários (users: []);
- reducers: contém as funções (reducers) que atualizam o estado. Cada reducer representa uma operação que pode ser realizada no estado;
- userSlice.actions:
    - Gera automaticamente as actions correspondentes aos reducers definidos;
    - Cada action possui um type baseado no nome do slice e do reducer (exemplo: cadastro/addUser).
- userSlice.reducer:

- Exporta o reducer do slice para ser utilizado na configuração da store.

Passo 6 – Integrar o Redux ao React

Com a Store configurada e os slices criados, precisamos conectar o Redux à aplicação React. Para isso, utilizamos o componente Provider do pacote react-redux.

Exemplo de integração no src/main.tsx:

```
import { createRoot } from "react-dom/client";
import App from "./App.tsx";
import { Provider } from "react-redux";
import { store } from "./redux/store.ts";

createRoot(document.getElementById("root")!).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

Explicação:

- Provider: torna a Store disponível para todos os componentes da aplicação React;
- store: a Store configurada anteriormente é passada como propriedade.

Passo 7 – Conectar componentes ao Redux

Para acessar o estado e despachar ações dentro dos componentes, utilizamos os hooks `useSelector` e `useDispatch` do react-redux.

Exemplo de componente conectado ao Redux (arquivo: src/App.tsx):

```
import { useDispatch, useSelector } from "react-redux";
import type { AppDispatch, RootState } from "./redux/store";
import { addUser, incrementAge, removeUser } from "./redux/slices/userSlice";
import type { User } from "./redux/slices/userSlice";
import { useState } from "react";

export default function App() {
  const users = useSelector((state: RootState) => state.userObject.users);
  const [id, setId] = useState(1);
  const [name, setName] = useState("");
  const [age, setAge] = useState("");

  const dispatch: AppDispatch = useDispatch();

  const handleSave = () => {
    const user: User = {
```

```
      id,
      name,
      age: parseInt(age),
    };
    dispatch(addUser(user));
    setId((prev: number) => prev + 1);
  };

  const handleAge = (id:number) => {
    dispatch( incrementAge(id) );
  };

  const handleRemove = (e: React.MouseEvent<HTMLSpanElement>, id: number) => {
    e.preventDefault();
    dispatch(removeUser(id));
  };

  return (
    <div>
      <div>
        <div>
          <label htmlFor="name">Nome</label>
          <input
            id="name"
            value={name}
            onChange={(e) => setName(e.target.value)}
          />
        </div>
        <div>
          <label htmlFor="age">Idade</label>
          <input
            id="age"
            value={age}
            onChange={(e) => setAge(e.target.value)}
          />
        </div>
        <div>
          <button onClick={handleSave}>Salvar</button>
        </div>
      </div>
      <ul>
        {users.map((user) => (
          <li key={user.id}>
            <span onClick={() => handleAge(user.id)} onContextMenu={(e) =>
handleRemove(e, user.id)}>
              {user.name} - {user.age}
            </span>
          </li>
        ))}
```

```
      </ul>
    </div>
  );
}
```

Execute → C:\Aula1Redux\front> npm run dev

Servidor rodando:

> front@0.0.0 dev

> vite

VITE v7.1.2 ready in 727 ms

- **Local: http://localhost:5173/**
- **Network: use --host to expose**
- **press h + enter to show help**